CS 3006
Parallel and Distributed Computing
Lecture 14

Danyal Farhat

FAST School of Computing

NUCES Lahore

Basic Communication Operations

Lecture's Agenda

- **Basic Communication Operations**
  - **Preliminaries**
  - **Assumptions for the Operations**
  - **One-to-All Broadcast & All-to-One Reduction**
    - ✓ **Linear Array or Ring**
    - ✓ **Matrix-Vector Multiplication (An Application)**
    - ✓ **Mesh Broadcast and Reduction**
    - ✓ **Balanced Binary Tree**
    - ✓ **Hypercube Broadcast (3-D Structure)**
    - ✓ **Cost Estimation**

Lecture's Agenda (Cont.)

- **All-to-All Broadcast and All-to-All Reduction**
  - **Linear Ring Broadcast**
  - **Linear Ring Reduction**
  - **All-to-All Broadcast on 2D Mesh**
  - **All-to-All Broadcast on HyperCube**
  - **Cost Estimation**
- **Exercise**
- **Summary**
- **Additional Resources**

Basic Communication Operations

Preliminaries

- **Exchanging the data is fundamental requirement for most of the parallel algorithms**
- **$t_s + mt_w$ - the simplified communication cost model :-**
  - **Over distributed memory infrastructure**
  - **Assuming the cut-through routing**
- **The chapter is about commonly used basic communication patterns over the different interconnections**
  - **We shall drive communication costs of these operations on different interconnections**

Basic Communication Operations (Cont.)

Assumptions for the Operations

- Interconnections support cut-through routing
- Communication time between any pair of nodes in the network is same (regardless of the number of intermediate nodes)
- Links are bi-directional
  - The directly connected nodes can simultaneously send messages of *m words* without any congestion
- Single-port communication model
  - A node can send on only one of its links at a time
  - A node can receive on only one of its links at a time
- However, a node can receive a message while sending another message at the same time on the same or a different link

Basic Communication Operations (Cont.)

One-to-All Broadcast

- A single process sends identical data to all other processes
  - Initially one process has data of *m* size.
  - After broadcast operation, each of the processes have own copy of the *m* size.

All-to-One Reduction

- Dual of one-to-all broadcast
- The m-sized data from all processes are combined through an associative operator, accumulated at a single destination process into one buffer of size *m*
- Applications: matrix-vector multiplication, shortest paths calculation, and inner vector product

Basic Communication Operations (Cont.)

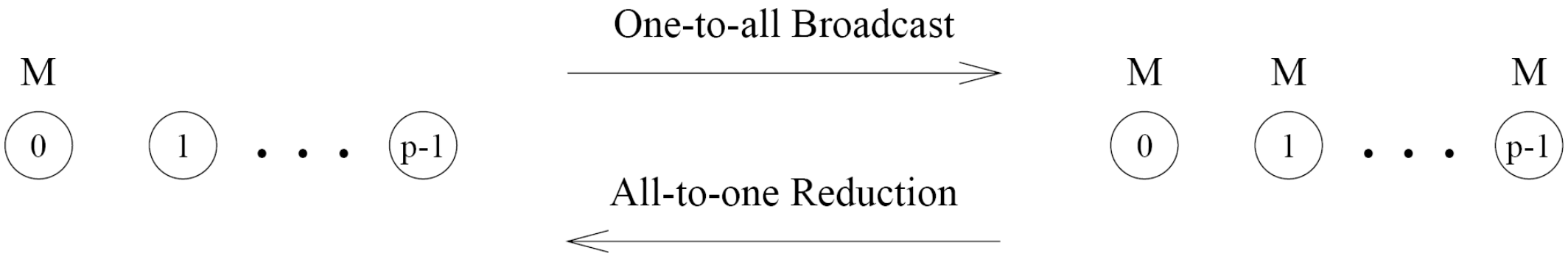


Figure 4.1 One-to-all broadcast and all-to-one reduction.

One-to-All Broadcast and All-to-One Reduction

Linear Array or Ring

- **Naïve solution**
  - **sequentially send *p* - 1 messages from the source to the other *p* - 1 processes**
    - ✓ **Bottlenecks, and underutilization of communication network**
  - **Solution?**

- **Recursive doubling**
  - **Source process sends the massage to another process**
  - **In next communication phase both the processes can simultaneously propagate the message**

One-to-All Broadcast and All-to-One Reduction

Linear Array or Ring (One Dimensional Structure)

- **Recursive Doubling Broadcast**
  - **Select destination such that no congestions occur**

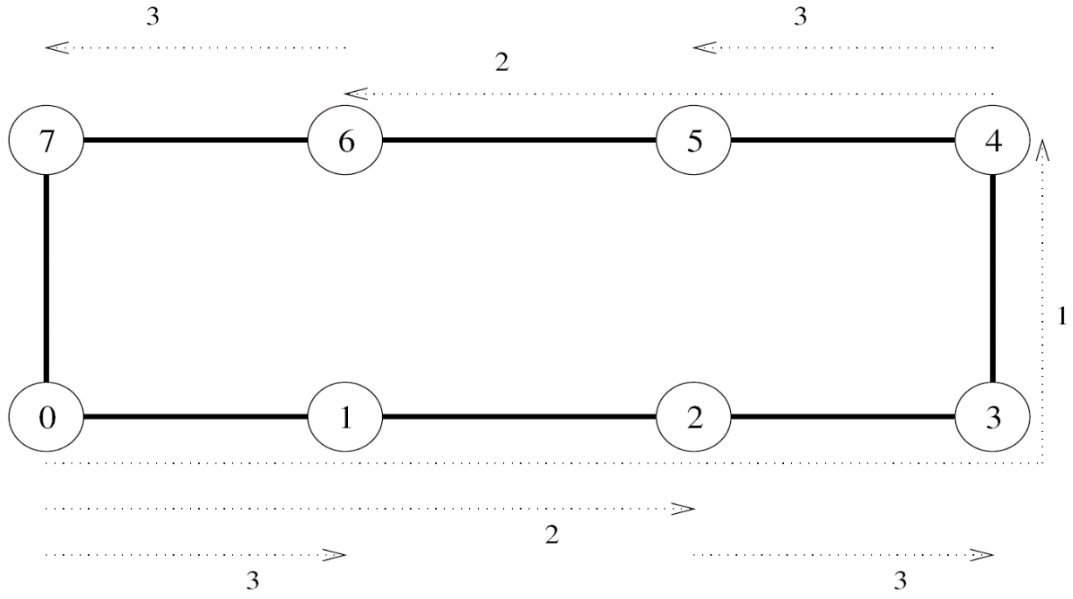


Figure 4.2 One-to-all broadcast on an eight-node ring. Node 0 is the source of the broadcast. Each message transfer step is shown by a numbered, dotted arrow from the source of the message to its destination. The number on an arrow indicates the time step during which the message is transferred.

One-to-All Broadcast and All-to-One Reduction

Linear Array or Ring (One Dimensional Structure)

- **Recursive Doubling Reduction**
  - **Odd will send message to even and associative operator will combine the results**

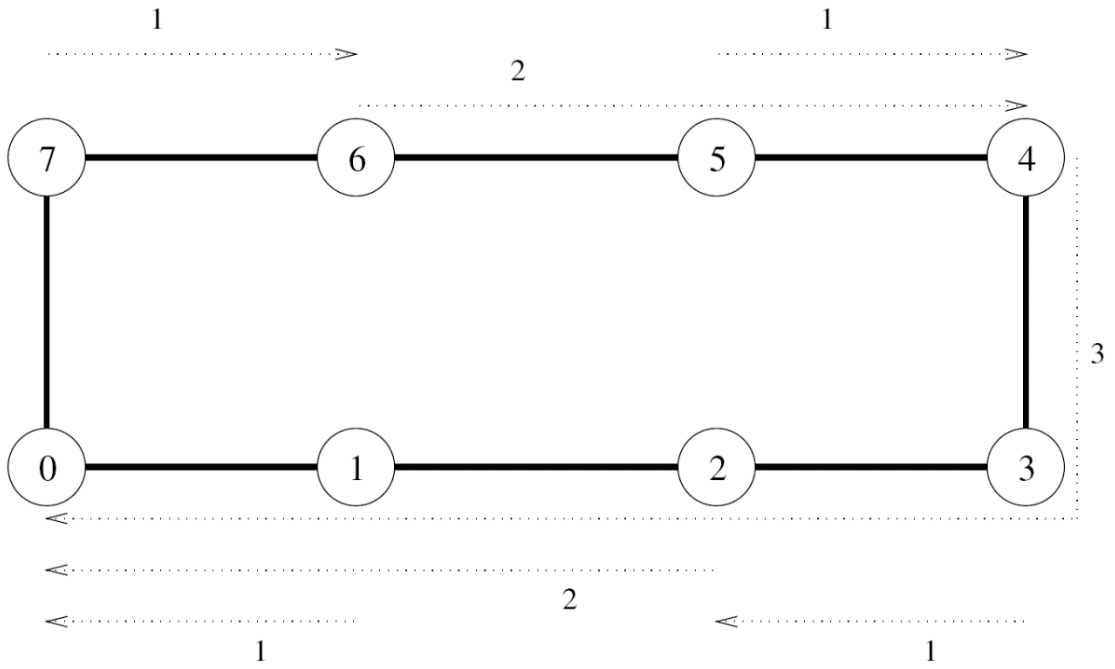


Figure 4.3 Reduction on an eight-node ring with node 0 as the destination of the reduction.

Matrix-Vector Multiplication (An Application)

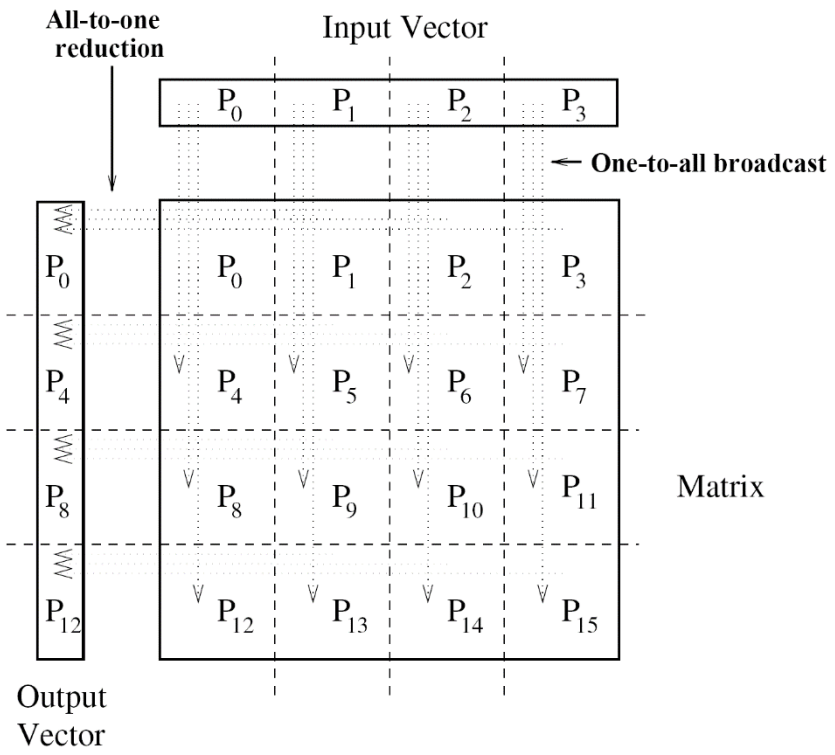


Figure 4.4 One-to-all broadcast and all-to-one reduction in the multiplication of a 4×4 matrix with a 4×1 vector.

Mesh Broadcast and Reduction

- **We can regard each row and column of a square mesh of *p* nodes as a linear array of nodes**
- **Communication algorithms on the mesh are simple extensions of their linear array counterparts**

- **Broadcast and Reduction**
  - **Two step breakdown:**
    - ✓ **The operation is performed along one by treating the row as linear array**
    - ✓ **Then the all the columns are treated similarly**

Mesh Broadcast and Reduction (Cont.)

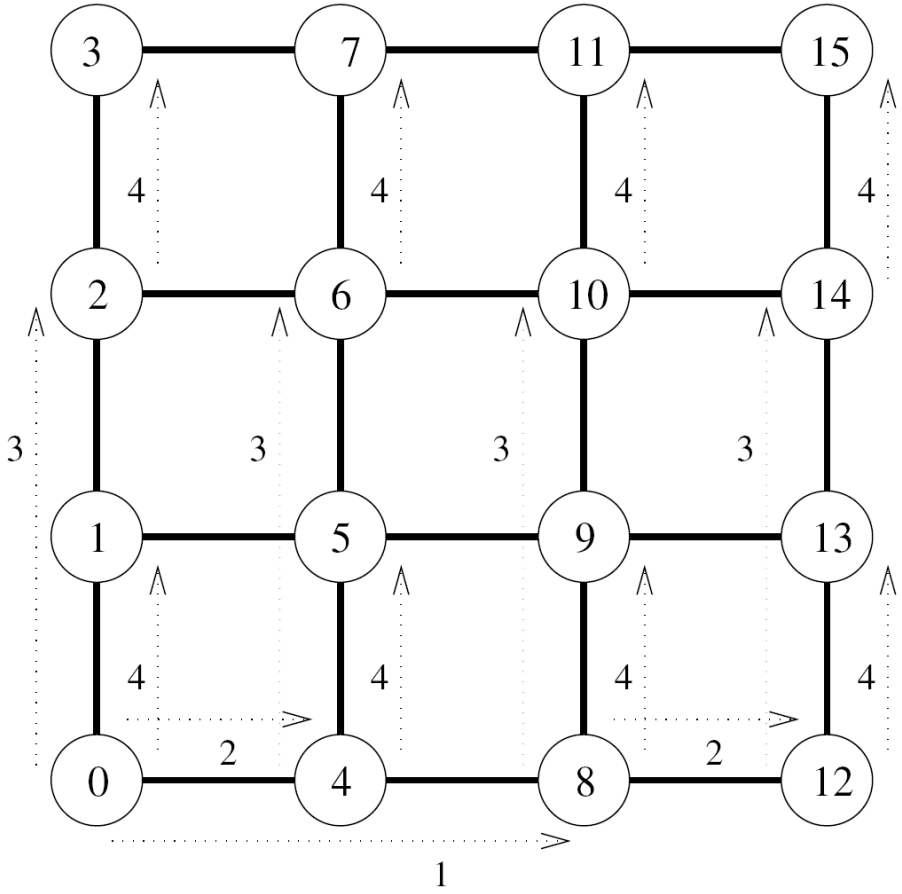


Figure 4.5 One-to-all broadcast on a 16-node mesh.

Balanced Binary Tree

- **Broadcast**

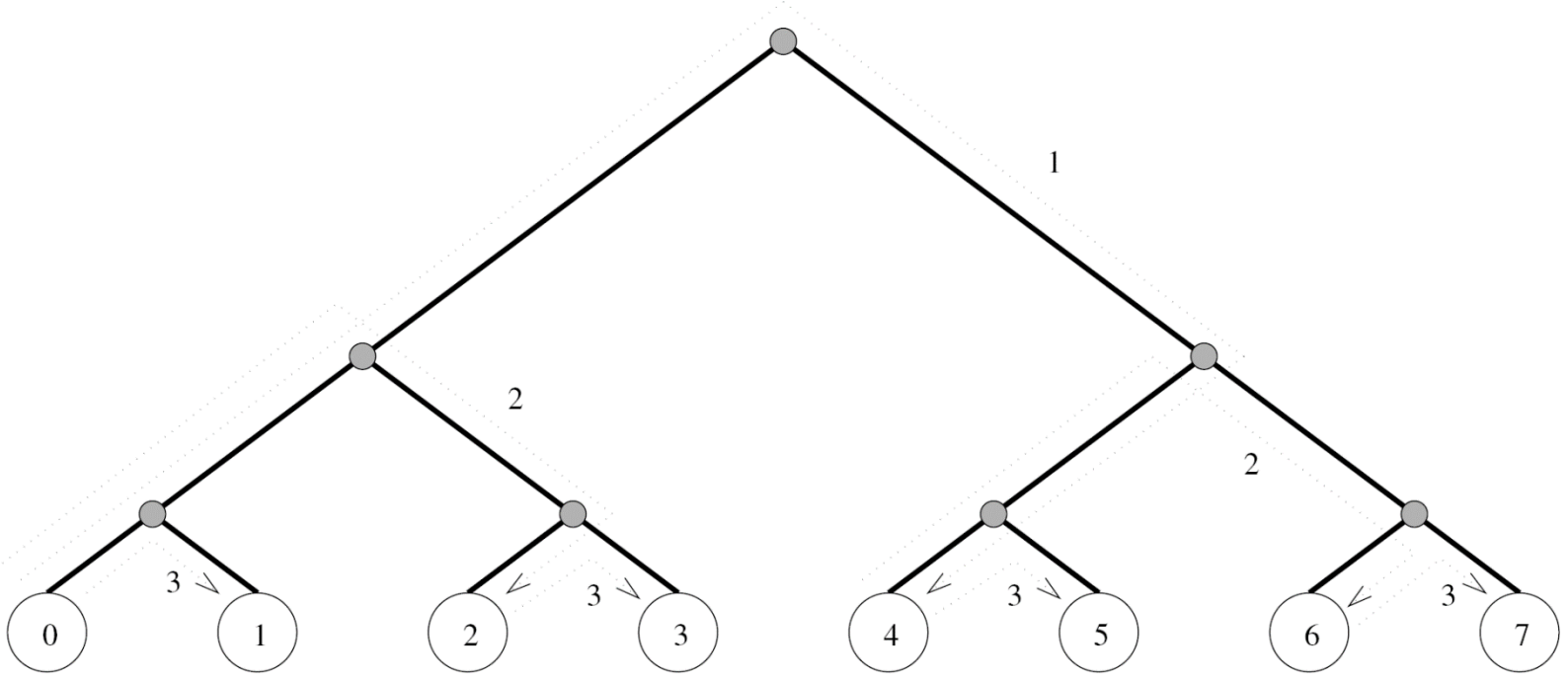


Figure 4.7 One-to-all broadcast on an eight-node tree.

Hypercube Broadcast (3-D Structure)

- **Source node first send data to one node in the highest dimension**
- **The communication successively proceeds along lower dimensions in the subsequent steps**
- **The algorithm is same as used for linear array**
  - **But, here changing order of dimension does not congest the network**
    - **i.e. can move from x-axis to z-axis or from z-axis to x-axis**

Hypercube Broadcast (Cont.)

- **Communication done along z-axis, then y-axis, and then x-axis**

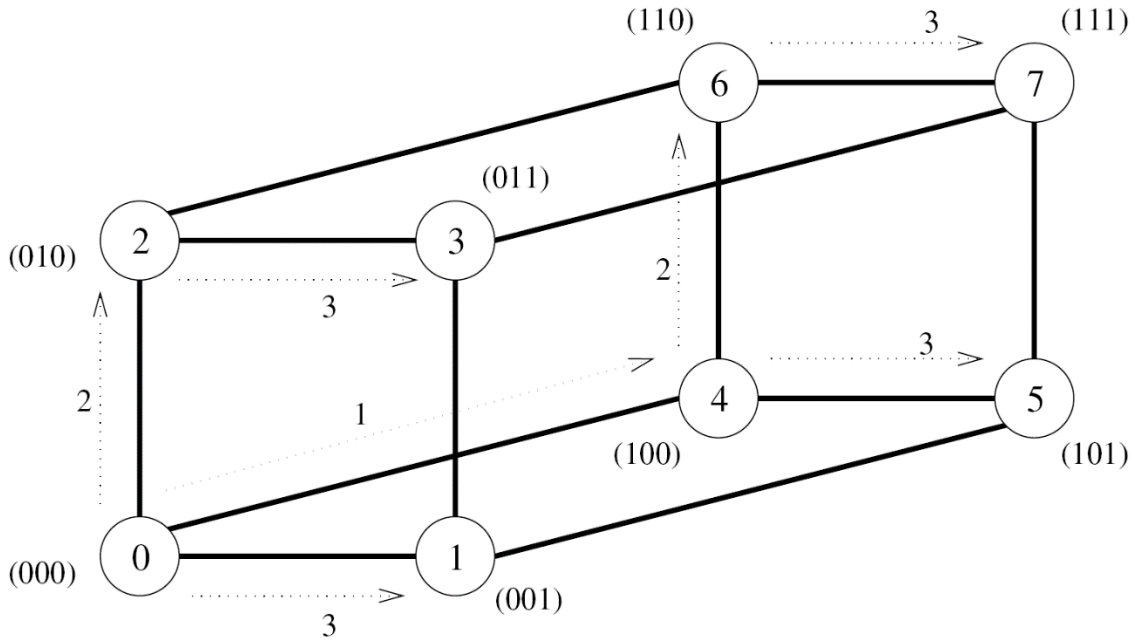


Figure 4.6 One-to-all broadcast on a three-dimensional hypercube. The binary representations of node labels are shown in parentheses.

Hypercube Broadcast General

- **Assigned Reading**

```
1.  procedure ONE_TO_ALL_BC(d, my_id, X)
2.  begin
3.      mask := 2^d − 1;                    /* Set all d bits of mask to 1 */
4.      for i := d − 1 downto 0 do          /* Outer loop */
5.          mask := mask XOR 2^i;           /* Set bit i of mask to 0 */
6.          if (my_id AND mask) = 0 then    /* If lower i bits of my_id are 0 */
7.              if (my_id AND 2^i) = 0 then
8.                  msg_destination := my_id XOR 2^i;
9.                  send X to msg_destination;
10.             else
11.                 msg_source := my_id XOR 2^i;
12.                 receive X from msg_source;
13.             endelse;
14.         endif;
15.     endfor;
16. end ONE_TO_ALL_BC
```

Algorithm 4.1 One-to-all broadcast of a message X from node 0 of a d-dimensional p-node hypercube ($d = \log p$). AND and XOR are bitwise logical-and and exclusive-or operations, respectively.

Hypercube Broadcast General (Cont.)

```
1.  procedure GENERAL_ONE_TO_ALL_BC(d, my_id, source, X)
2.  begin
3.      my_virtual_id := my_id XOR source;
4.      mask := 2^d − 1;
5.      for i := d − 1 downto 0 do      /* Outer loop */
6.          mask := mask XOR 2^i;    /* Set bit i of mask to 0 */
7.          if (my_virtual_id AND mask) = 0 then
8.              if (my_virtual_id AND 2^i) = 0 then
9.                  virtual_dest := my_virtual_id XOR 2^i;
10.                 send X to (virtual_dest XOR source);
        /* Convert virtual_dest to the label of the physical destination */
11.             else
12.                 virtual_source := my_virtual_id XOR 2^i;
13.                 receive X from (virtual_source XOR source);
        /* Convert virtual_source to the label of the physical source */
14.             endelse;
15.     endfor;
16. end GENERAL_ONE_TO_ALL_BC
```

Algorithm 4.2 One-to-all broadcast of a message X initiated by $source$ on a d-dimensional hypothetical hypercube. The AND and XOR operations are bitwise logical operations.

Hypercube Reduction

- **Assigned Reading**

```
1.   procedure ALL_TO_ONE_REDUCE(d, my_id, m, X, sum)
2.   begin
3.       for j := 0 to m − 1 do sum[j] := X[j];
4.       mask := 0;
5.       for i := 0 to d − 1 do
             /* Select nodes whose lower i bits are 0 */
6.           if (my_id AND mask) = 0 then
7.               if (my_id AND 2^i) ≠ 0 then
8.                   msg_destination := my_id XOR 2^i;
9.                   send sum to msg_destination;
10.              else
11.                  msg_source := my_id XOR 2^i;
12.                  receive X from msg_source;
13.                  for j := 0 to m − 1 do
14.                      sum[j] := sum[j] + X[j];
15.              endelse;
16.          mask := mask XOR 2^i;    /* Set bit i of mask to 1 */
17.      endfor;
18.  end ALL_TO_ONE_REDUCE
```

Cost Estimation

- **Broadcast needs log(p) point-to-point simple message transfer steps.**
- **Message size of each transfer is m**
- **Time foreach of the transfers is: $t_s + mt_w$**

- **Hence cost for log(p) transfers = T = $(t_s + mt_w)\ log\ p$**

All-to-All Broadcast and All-to-All Reduction

All-to-All Broadcast

- A generalization to one-to-all broadcast
- Every process broadcasts m-word message
  - The broadcast-message for each of the processes can be different than others

All-to-All Reduction

- Dual of all-to-all broadcast
- Each node is the destination of an all-to-one reduction out of total P reductions
- Applications in different matrix operations including matrix multiplication and matrix-vector multiplication

All-to-All Broadcast and All-to-All Reduction (Cont.)

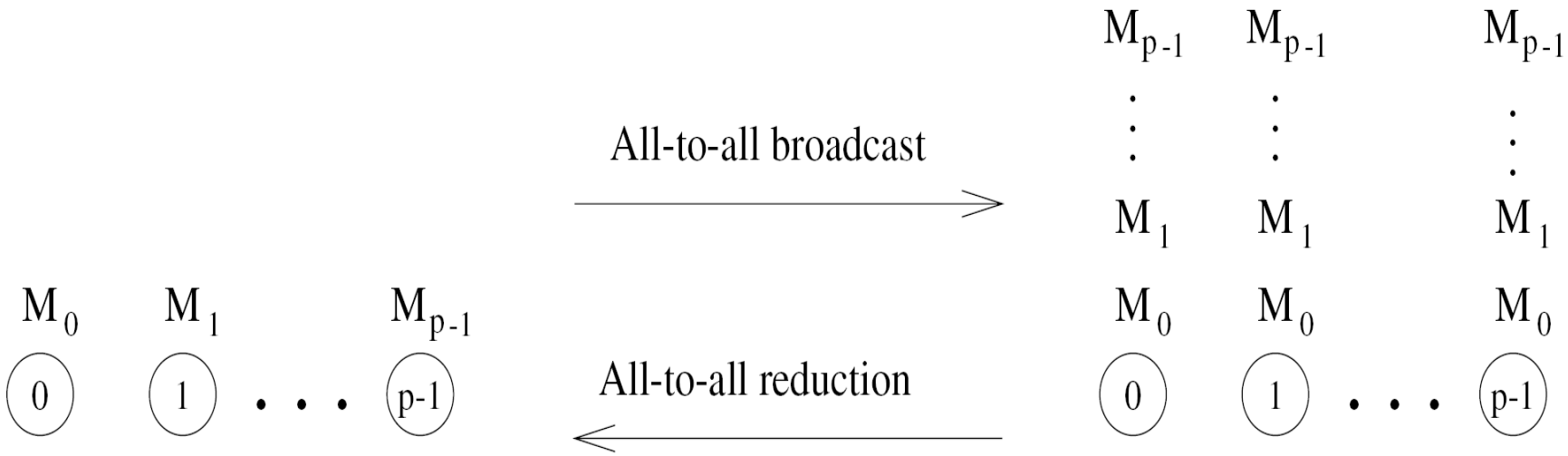


Figure 4.8 All-to-all broadcast and all-to-all reduction.

- A naïve Broadcast method may be performing **P** one-to-all broadcasts. This will result **P(log(p)(t(s) + mt(w)))** communication time.

All-to-All Broadcast and All-to-All Reduction (Cont.)

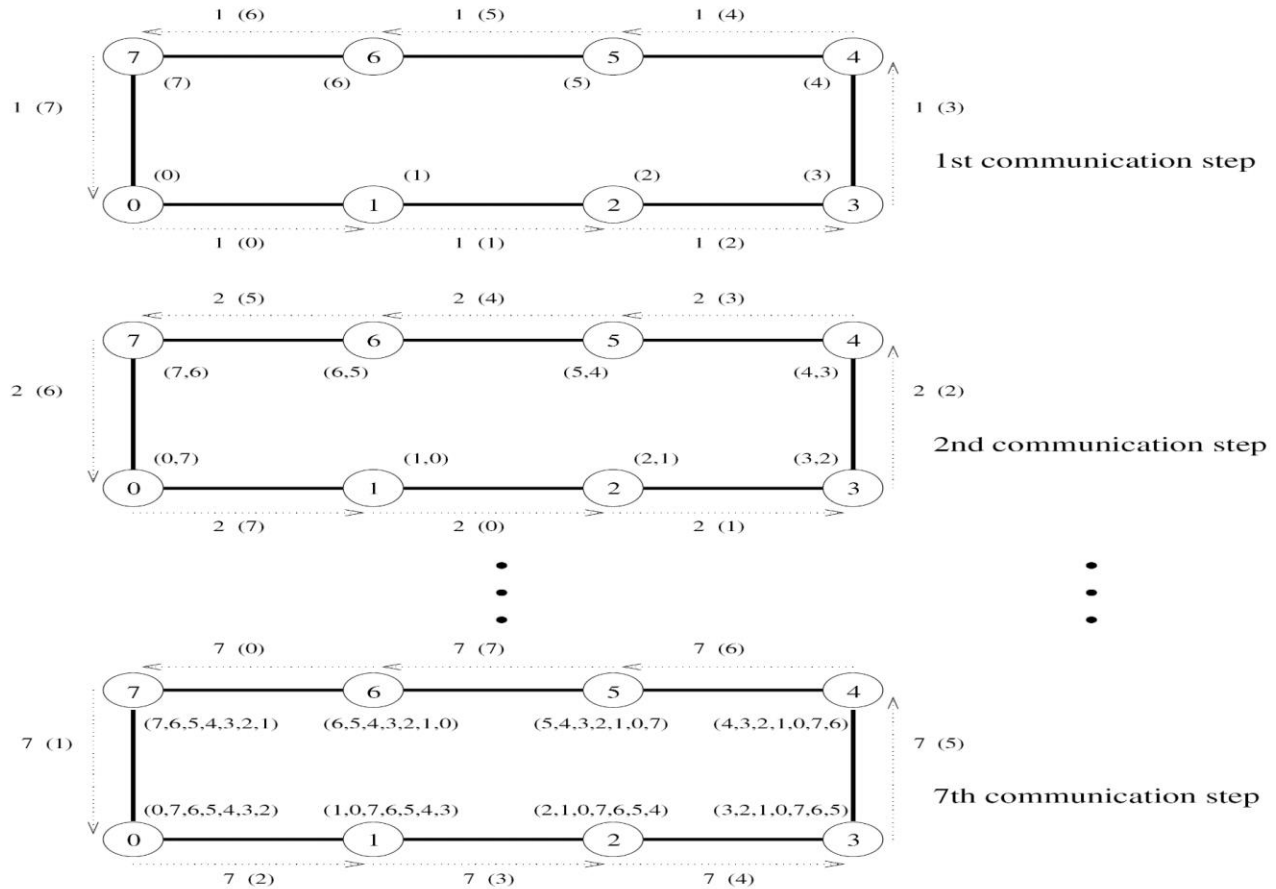


Figure 4.9 All-to-all broadcast on an eight-node ring. The label of each arrow shows the time step and, within parentheses, the label of the node that owned the current message being transferred before the beginning of the broadcast. The number(s) in parentheses next to each node are the labels of nodes from which data has been received prior to the current communication step. Only the first, second, and last communication steps are shown.

Linear Ring Broadcast (Assigned Reading)

```
1.   procedure ALL_TO_ALL_BC_RING(my_id, my_msg, p, result)
2.   begin
3.       left := (my_id − 1) mod p;
4.       right := (my_id + 1) mod p;
5.       result := my_msg;
6.       msg := result;
7.       for i := 1 to p − 1 do
8.           send msg to right;
9.           receive msg from left;
10.          result := result ∪ msg;
11.      endfor;
12.  end ALL_TO_ALL_BC_RING
```

Algorithm 4.4 All-to-all broadcast on a p-node ring.

All-to-All Reduction

Linear Array or Ring

- **Reduction**
  - **Draw an All-to-All Broadcast on a P-node linear ring**
  - **Reverse the directions in each of the step without changing message**
  - **After each communication step, combine messages having same broadcast destination with associative operator**

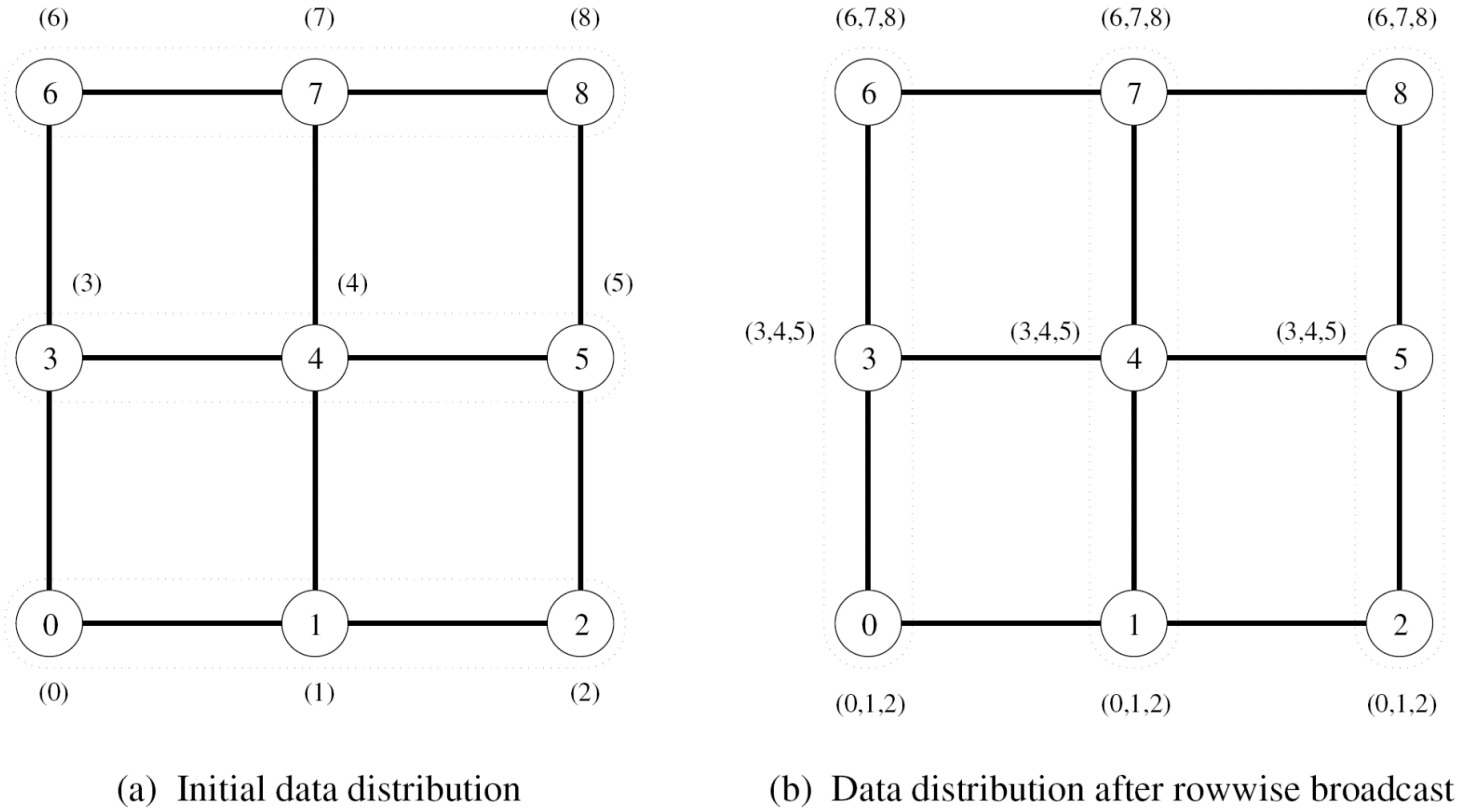
- **Now, Its your turn to draw?**
  - **Draw an All-to-All Broadcast on a 4-node linear ring**
  - **Reverse the directions and combine the results using ‘SUM’**

Linear Ring Reduction (Assigned Reading)

```
1.   procedure ALL_TO_ALL_RED_RING(my_id, my_msg, p, result)
2.   begin
3.       left := (my_id − 1) mod p;
4.       right := (my_id + 1) mod p;
5.       recv := 0;
6.       for i := 1 to p − 1 do
7.           j := (my_id + i) mod p;
8.           temp := msg[j] + recv;
9.           send temp to left;
10.          receive recv from right;
11.      endfor;
12.      result := msg[my_id] + recv;
13.  end ALL_TO_ALL_RED_RING
```

Algorithm 4.5 All-to-all reduction on a p-node ring.

All-to-All Broadcast on 2D Mesh



(a) Initial data distribution

(b) Data distribution after rowwise broadcast

Figure 4.10 All-to-all broadcast on a 3×3 mesh. The groups of nodes communicating with each other in each phase are enclosed by dotted boundaries. By the end of the second phase, all nodes get (0,1,2,3,4,5,6,7) (that is, a message from each node).

All-to-All Broadcast on 2D Mesh Algorithm (Assigned Reading)

```
1.   procedure ALL_TO_ALL_BC_MESH(my_id, my_msg, p, result)
2.   begin

/* Communication along rows */
3.       left := my_id − (my_id mod √p) + (my_id − 1) mod √p;
4.       right := my_id − (my_id mod √p) + (my_id + 1) mod √p;
5.       result := my_msg;
6.       msg := result;
7.       for i := 1 to √p − 1 do
8.           send msg to right;
9.           receive msg from left;
10.          result := result ∪ msg;
11.      endfor;

/* Communication along columns */
12.      up := (my_id − √p) mod p;
13.      down := (my_id + √p) mod p;
14.      msg := result;
15.      for i := 1 to √p − 1 do
16.          send msg to down;
17.          receive msg from up;
18.          result := result ∪ msg;
19.      endfor;
20.  end ALL_TO_ALL_BC_MESH
```

Algorithm 4.6 All-to-all broadcast on a square mesh of p nodes.

All-to-All Broadcast on HyperCube

- Along x-axis then y-axis then z-axis

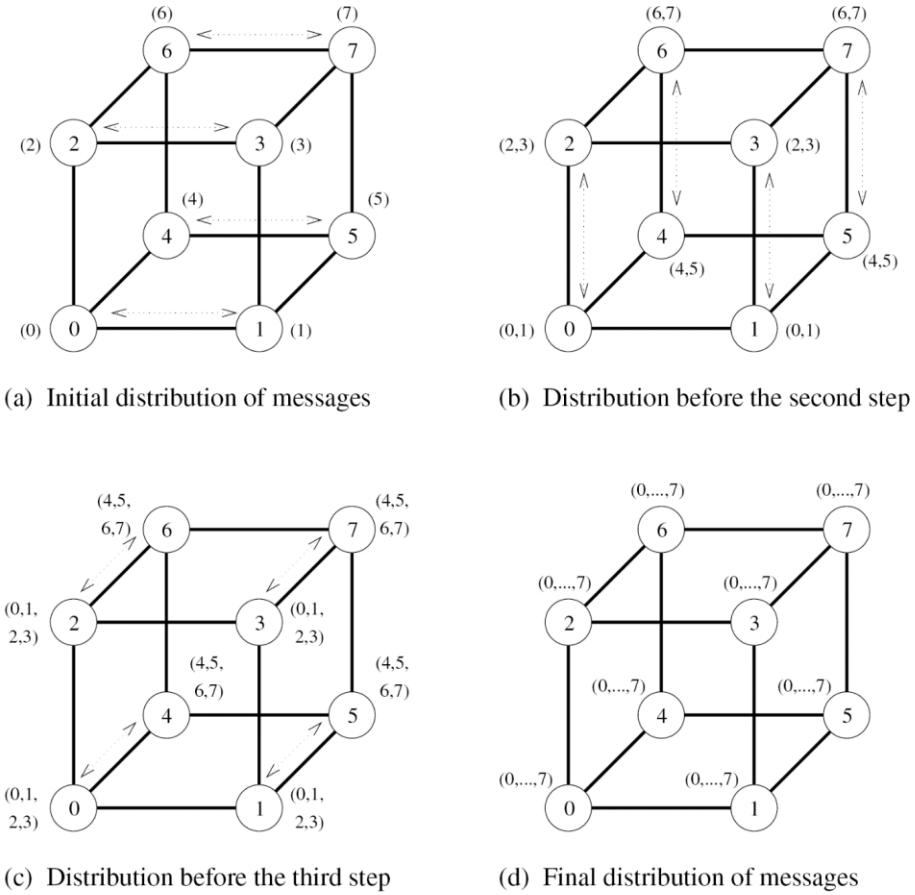


Figure 4.11 All-to-all broadcast on an eight-node hypercube.

All-to-All Broadcast on HyperCube..Algorithm (Assigned Reading)

```
1.   procedure ALL_TO_ALL_BC_HCUBE(my_id, my_msg, d, result)
2.   begin
3.       result := my_msg;
4.       for i := 0 to d − 1 do
5.           partner := my_id XOR 2^i;
6.           send result to partner;
7.           receive msg from partner;
8.           result := result ∪ msg;
9.       endfor;
10.  end ALL_TO_ALL_BC_HCUBE
```

Algorithm 4.7 All-to-all broadcast on a d-dimensional hypercube.

All-to-All Broadcast and All-to-All Reduction

Cost Estimation

- **Different on each infrastructure**
- **Linear Ring**
  - $T = (t_s + mt_w)(P - 1)$
- **Mesh**
  - **Total time for All-to-All broadcast in the first phase**
    - ✓ $T(first\ phase) = (t_s + mt_w)(\sqrt{p} - 1)$
  - **Total time for the second phase (note here m= $\sqrt{p}$.m)**
    - ✓ $T(Second\ phase) = (t_s + (\sqrt{p})mt_w)(\sqrt{p} - 1)$
- **So, Total time=** $2t_s(\sqrt{p} - 1) + mt_w(p - 1)$

All-to-All Broadcast and All-to-All Reduction (Cont.)

Cost Estimation (Cont.)

- **Different on each infrastructure**
- **Hypercube (broadcast)**
  - **Communication in for 1st step:** $(t_s + mt_w)$
  - **Communication in for 2nd step:** $(t_s + 2mt_w)$
  - **Communication in for ith step:** $(t_s + 2^{i-1}mt_w)$

- **Total Cost=** $\sum_{i=1}^{log(p)} (t_s + 2^{i-1}mt_w)$

All-to-All Broadcast and All-to-All Reduction (Cont.)

Cost Estimation (Cont.)

- Total Cost= $\sum_{i=1}^{log(p)}(t_s+2^{i-1}mt_w)$
- Simplify the equation
- HINT: [$x^0 + x^1$ + ……+ $x^n = \frac{x^{n+1}-1}{x-1}$]
- Answer $T = (t_s \, log \, p + mt_w(p-1))$

Summary

- **Basic Communication Operations**

  - **Preliminaries**
    - ✓ **Exchanging the data is fundamental requirement for most of the parallel algorithms**

  - **Assumptions for the Operations**
    - ✓ **Interconnections support cut-through routing**
    - ✓ **Communication time between any pair of nodes in the network is same**
    - ✓ **Links are bi-directional**
    - ✓ **Single-port communication model**

Summary (Cont.)

- **One-to-All Broadcast & All-to-One Reduction**

  - **One-to-All Broadcast - A single process sends identical data to all other processes**

  - **All-to-One Reduction - The m-sized data from all processes are combined through an associative operator, accumulated at a single destination process into one buffer of size m**
    - ✓ **Linear Array or Ring**
    - ✓ **Matrix-Vector Multiplication (An Application)**
    - ✓ **Mesh Broadcast and Reduction**
    - ✓ **Balanced Binary Tree**
    - ✓ **Hypercube Broadcast (3-D Structure)**
    - ✓ **Cost Estimation**

Summary (Cont.)

- **All-to-All Broadcast and All-to-All Reduction**
  - **All-to-All Broadcast - Every process broadcasts m-word message**
  - **All-to-All Reduction - Each node is the destination of an all-to-one reduction out of total P reductions**
    - ✓ **Linear Ring Broadcast**
    - ✓ **Linear Ring Reduction**
    - ✓ **All-to-All Broadcast on 2D Mesh**
    - ✓ **All-to-All Broadcast on HyperCube**
    - ✓ **Cost Estimation**
- **Exercise**

Additional Resources

- **Introduction to Parallel Computing by Ananth Grama and Anshul Gupta**

  - **Chapter 4: Basic Communication Operations**

Questions?

Class Activity